



Instalação em Servidor Novo

Para quem instala

Documentação técnica e operacional do sistema RedeApoio

Gerado em 04 de agosto de 2026

Origem: docs/01-instalacao-servidor-novo.md

Este PDF é gerado automaticamente a partir da documentação do repositório. A versão sempre atual está em **docs/01-instalacao-servidor-novo.md**, no GitHub — em caso de divergência, vale o que está lá.

1 — Instalação do zero num servidor Ubuntu novo

Passo a passo completo para colocar o RedeApoio no ar num Ubuntu limpo: Docker, Swarm, Traefik, Portainer, banco e aplicação. Ao final você tem o sistema funcionando em `https://seu-dominio` com certificado SSL automático.

Tempo estimado: 40 a 60 minutos, sendo boa parte só espera.

O que você precisa ter em mãos antes de começar

Item	Onde consegue
IP público do servidor e acesso SSH root (ou sudo)	painel da hospedagem (Hostinger, Contabo, DigitalOcean...)
Ubuntu 22.04 ou 24.04 LTS, mínimo 2 GB de RAM	idem
Acesso ao painel de DNS do domínio	onde o domínio foi registrado
Dados SMTP (host, porta, usuário, senha)	painel de e-mail da Hostinger
Backup do banco do servidor atual, se for migração	ver Backup, Restauração e Migração

Requisito de RAM. As stacks reservam no máximo 512 MB (Postgres) + 512 MB (app) + 512 MB (Portainer) + 256 MB (Traefik). Com 2 GB o servidor roda confortável. Com 1 GB roda, mas o `docker build` pode ficar sem memória — nesse caso crie swap antes (passo 1.4).

Passo 1 — Preparar o Ubuntu

Conecte via SSH:

```
ssh root@SEU_IP_AQUI
```

1.1 Atualizar o sistema

```
apt update && apt upgrade -y
```

1.2 Definir fuso horário

Importante: as datas de desativação de contrato e os carimbos de aceite do termo LGPD usam o relógio do servidor.

```
timedatectl set-timezone America/Campo_Grande
timedatectl          # confira se mostra -04 e "System clock synchronized: yes"
```

Ajuste para `America/Sao_Paulo` se a operação for no fuso de Brasília.

1.3 Firewall

Só três portas precisam ficar abertas. **Abra o SSH primeiro** — se você ativar o `ufw` sem liberar a 22, perde o acesso ao servidor.

```
ufw allow 22/tcp      # SSH – SEMPRE primeiro
ufw allow 80/tcp      # HTTP (validação do certificado)
ufw allow 443/tcp     # HTTPS
ufw --force enable
ufw status
```

O Postgres **não** fica exposto: ele vive numa rede Docker marcada como `internal: true` e não tem porta publicada no host. Isso é intencional — não abra a 5432.

1.4 Swap (só se o servidor tiver 2 GB de RAM ou menos)

```
fallocate -l 2G /swapfile && chmod 600 /swapfile
mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
free -h
```

Passo 2 — Instalar o Docker

Use o repositório oficial. O Docker que vem no `apt` do Ubuntu é antigo e não tem os recursos de Swarm que a stack usa.

```
apt install -y ca-certificates curl gnupg
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
  > /etc/apt/sources.list.d/docker.list

apt update
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Confira:

```
docker --version          # espera-se algo como "Docker version 27.x"
docker run --rm hello-world
```

Passo 3 — Inicializar o Swarm e criar a rede

O sistema roda em **Docker Swarm**, não em Docker Compose comum. É o Swarm que dá os recursos que a stack usa: reinício automático, limites de memória, `docker service scale` e as labels que o Traefik lê.

```
# Troque SEU_IP_AQUI pelo IP público do servidor
docker swarm init --advertise-addr SEU_IP_AQUI
```

Se o comando reclamar de múltiplos endereços, é porque o servidor tem mais de uma interface — passar o IP explicitamente, como acima, resolve.

Crie a rede compartilhada. **Este nome exato** (`network_public`) é o que aparece nos três arquivos de stack — se mudar aqui, tem que mudar em todos:

```
docker network create --driver=overlay --attachable network_public
docker network ls | grep network_public
```

Passo 4 — Apontar o DNS (faça agora, antes do Traefik)

O Let's Encrypt só emite o certificado se conseguir acessar o domínio e confirmar que ele aponta para este servidor. **Se o DNS não estiver pronto, o site sobe sem HTTPS** e você vai precisar forçar uma nova

tentativa depois.

No painel de DNS do domínio, crie dois registros do tipo **A**:

Tipo	Nome	Valor	TTL
A	app (ou o subdomínio do sistema)	IP do servidor	300
A	painel (para o Portainer)	IP do servidor	300

Espere a propagação e confirme **do próprio servidor**:

```
apt install -y dnsutils
dig +short app.seudominio.com.br
dig +short painel.seudominio.com.br
```

Os dois precisam devolver o IP do servidor novo. Costuma levar de 5 a 30 minutos. **Não siga adiante enquanto não devolverem.**

Migração com o domínio já em uso no servidor antigo: não repontar o DNS ainda. Use um subdomínio temporário (ex: `novo.seudominio.com.br`) para montar e testar tudo, e só troque o DNS definitivo no final. O procedimento está em [Backup, Restauração e Migração](#).

Passo 5 — Subir o Traefik

O Traefik é o porteiro: recebe todo tráfego nas portas 80/443, decide qual container responde por cada domínio e cuida sozinho dos certificados SSL.

Traga o repositório para o servidor:

```
apt install -y git
mkdir -p /opt && cd /opt
git clone -b 1.0 https://github.com/adrianasouza77/Redeapio.git redeapoiopolitico
cd /opt/redeapoiopolitico
```

Repositório privado? O `git clone` vai pedir usuário e senha — use um **Personal Access Token** do GitHub no lugar da senha (Settings → Developer settings → Personal access tokens).

Suba a stack:

```
export ACME_EMAIL=seuemail@dominio.com.br
docker stack deploy -c infra/traefik-stack.yml traefik
```

Acompanhe até aparecer `1/1`:

```
docker service ls
docker service logs -f traefik_traefik
```

Nos logs, procure por linhas de `certificate obtained`. Erros de `acme: error presenting token` quase sempre significam DNS ainda não propagado ou porta 80 fechada.

Passo 6 — Subir o Portainer

Este é o único deploy feito por linha de comando — daqui em diante tudo passa a ser pelo painel.

```
export PORTAINER_DOMAIN=painel.seudominio.com.br
docker stack deploy -c infra/portainer-stack.yml portainer
docker service ls | grep portainer
```

Abra `https://painel.seudominio.com.br` no navegador.

 **Você tem 5 minutos.** Por segurança, o Portainer bloqueia a criação do usuário administrador se ninguém fizer isso logo após ele subir. Se aparecer a mensagem de tempo esgotado, rode `docker service update --force portainer_portainer` e recarregue a página.

Crie o usuário admin com uma **senha forte e guardada em lugar seguro** — quem entra aqui controla todos os containers e vê todas as senhas das stacks.

Na tela seguinte, escolha **"Get Started"** / ambiente local.

Passo 7 — Gerar as senhas do sistema

Ainda no SSH, gere os dois segredos. Copie a saída para um gerenciador de senhas **antes** de continuar:

```
echo "DB_PASSWORD = $(openssl rand -hex 24)"
echo "JWT_SECRET = $(openssl rand -hex 32)"
```

O que cada um faz:

- `DB_PASSWORD` — senha do Postgres. Ela é gravada dentro do volume do banco no primeiro boot. Trocar depois exige entrar no banco e alterar o papel manualmente; não é algo que se muda editando a variável.
- `JWT_SECRET` — assina os cookies de sessão. Se mudar, todo mundo é deslogado e precisa entrar de novo (as senhas continuam valendo).

Vindo de outro servidor? Se quiser que as sessões abertas continuem valendo, use o **mesmo** `JWT_SECRET` do servidor antigo. As senhas dos usuários migram de qualquer jeito — elas são hashes bcrypt guardados no banco, não dependem dessas variáveis.

Passo 8 — Buildar a imagem da aplicação

O Swarm **não builda imagens** a partir do compose — ele só baixa imagens prontas. Por isso a imagem é construída localmente, no servidor:

```
cd /opt/redeapoioipolitico
bash build.sh
```

A saída deve terminar com `redeapoio-app latest ...`. Como o serviço ainda não existe, o script apenas lembra os próximos passos.

Passo 9 — Subir a stack do RedeApoio pelo Portainer

No painel: **Stacks** → **Add stack**.

- Name:** `redeapoio` (*exatamente assim — os scripts de backup e o `build.sh` procuram os serviços pelos nomes `redeapoio_redeapoio-app` e `redeapoio_redeapoio-postgres`*)
- Build method:** Web editor
- Cole o conteúdo de `docker-compose.yml` (raiz do repositório)
- Em **Environment variables**, adicione uma a uma:

Nome	Valor
<code>DOMAIN</code>	<code>app.seudominio.com.br</code>
<code>DB_PASSWORD</code>	a senha gerada no passo 7
<code>JWT_SECRET</code>	o segredo gerado no passo 7
<code>SMTP_HOST</code>	<code>smtp.hostinger.com</code>
<code>SMTP_PORT</code>	<code>587</code>
<code>SMTP_USER</code>	<code>naoresponda@seudominio.com.br</code>
<code>SMTP_PASS</code>	senha da caixa de e-mail
<code>SMTP_FROM</code>	RedeApoio <naoresponda@seudominio.com.br>

5. Deploy the stack

As variáveis SMTP são opcionais. Sem elas o sistema funciona, mas o botão "Esqueci minha senha" para de enviar e-mail silenciosamente — a redefinição passa a depender do candidato ou do administrador gerar a senha na mão.

Não preencha `SUPABASE_URL` / `SUPABASE_SERVICE_KEY`. Elas existiam só para a importação única do Supabase, feita em julho de 2025, e não têm mais uso.

O que acontece sozinho no primeiro boot

O container do app aplica o schema do banco por conta própria: ele lê `backend/migrations/001_init.sql` e executa, repetindo a tentativa a cada 3 segundos (até 20 vezes) enquanto o Postgres ainda estiver subindo.

Não existe passo manual de criação de tabelas.

Acompanhe:

```
docker service logs -f redeapoio_redeapoio-app
```

Você deve ver `Migrações aplicadas com sucesso.` seguido de `RedeApoio backend rodando na porta 3000`.

Passo 10 — Criar o primeiro administrador

O banco começa vazio: não existe nenhum usuário e não há tela pública de cadastro de administrador. A primeira conta é criada direto no banco.

```
# 1. Gere o hash bcrypt de uma senha que você escolher
docker exec -it $(docker ps -q -f name=redeapoio_redeapoio-app) \
  node -e "require('bcryptjs').hash('TROQUE-ESTA-SENHA', 10).then(console.log)"
```

Copie o hash (começa com `$2a$10$` ou `$2b$10$`) e use na linha abaixo:

```
# 2. Crie o usuário admin
docker exec -it $(docker ps -q -f name=redeapoio_redeapoio-postgres) \
  psql -U redeapoio -d redeapoio -c \
  "INSERT INTO usuarios (nome, login, senha_hash, perfil)
  VALUES ('Administrador', 'admin', 'COLE_O_HASH_AQUI', 'admin');"
```

Entre em `https://app.seudominio.com.br`, escolha a aba **Administrador**, use `admin` e a senha que você escolheu. Troque a senha em **Minha Conta** depois.

Se você está migrando dados de outro servidor, pule este passo. As contas vêm todas no backup, com as senhas que já eram usadas. Vá para [Backup, Restauração e Migração](#).

Passo 11 — Conferir se está tudo certo

Verificação	Comando / ação	Resultado esperado
Serviços no ar	<code>docker service ls</code>	1/1 em todos
API respondendo	<code>curl -s https://app.seudominio.com.br/api/health</code>	<code>{"ok":true}</code>
Certificado válido	abrir o site no navegador	cadeado, sem aviso
HTTP redireciona	<code>curl -I http://app.seudominio.com.br</code>	301 para https
Banco com as tabelas	ver comando abaixo	3 tabelas
Login funciona	entrar com a conta admin	painel abre
Reinício automático	<code>reboot</code> , esperar 2 min, abrir o site	volta sozinho

```
docker exec $(docker ps -q -f name=redeapoio_redeapoio-postgres) \
  psql -U redeapoio -d redeapoio -c '\dt'
# esperado: usuarios, apoiadores, termos_aceite
```

O teste de reinício não é opcional — é ele que confirma que o sistema volta sozinho depois de uma queda de energia ou manutenção da hospedagem.

Passo 12 — Ativar o backup automático

Não considere a instalação terminada sem isto. Diferente do Supabase, um Postgres self-hosted não faz backup sozinho.

```
chmod +x /opt/redeapoiopolitico/scripts/*.sh
mkdir -p /var/backups/redeapoio

# Rode uma vez na mão para confirmar que funciona
bash /opt/redeapoiopolitico/scripts/backup-db.sh

# Agende para todo dia às 3h
crontab -e
```

Adicione a linha:

```
0 3 * * * /opt/redeapoiopolitico/scripts/backup-db.sh >> /var/log/redeapoio-backup.log 2>&1
```

No dia seguinte, confirme:

```
ls -lh /var/backups/redeapoio/
tail -20 /var/log/redeapoio-backup.log
```

Configure também a cópia para fora do servidor — as duas opções estão comentadas no fim do `scripts/backup-db.sh`. Backup que mora no mesmo servidor do banco não protege contra o cenário mais provável de perda total: o servidor inteiro sumir.

Resolvendo problemas comuns

O site não abre / "connection refused"

```
docker service ls                # tudo 1/1?
docker service ps redeapoio_redeapoio-app --no-trunc # erro na coluna ERROR?
docker service logs --tail 100 redeapoio_redeapoio-app
```

Certificado inválido ou "Traefik default cert"

Quase sempre é DNS. Confirme com `dig +short app.seudominio.com.br` **de dentro do servidor** e veja os logs do Traefik:

```
docker service logs traefik_traefik | grep -i acme
```

Se você errou o domínio e tentou muitas vezes, pode ter batido no limite do Let's Encrypt (5 certificados iguais por semana). Nesse caso, espere ou use um subdomínio diferente.

O app fica reiniciando

```
docker service logs --tail 200 redeapoio_redeapoio-app
```

- `JWT_SECRET` não definido → a variável não foi salva na stack; edite e atualize a stack de novo.
- `Postgres` ainda não disponível repetindo mais de 20 vezes → `DB_PASSWORD` diferente da que inicializou o volume. Se o banco ainda estiver vazio, a saída é apagar o volume e recomeçar:

```
docker stack rm redeapoio && sleep 15
docker volume rm redeapoio_redeapoio_pgdata
# suba a stack de novo com a senha certa
```

⚠ Isso apaga os dados. Só faça com o banco vazio ou com backup na mão.

`bash build.sh` **falha por falta de memória**

Crie swap (passo 1.4) e tente de novo.

Próximos documentos

- [Backup, Restauração e Migração](#) — levar os dados do servidor antigo
- [Manual do Dono do Sistema](#) — operar e alterar o sistema no dia a dia
- [Rotinas de Manutenção e Correção](#) — o que checar toda semana